Largest Rectangle Under Skyline

AlgoExpert · Hard · Stacks

The problem

- Largest rectangle area within adjacent buildings of width 1.
- Monotonic increasing stack of indices.

Intuition

- A monotonic stack finds, for each building, how far it extends left and right before a shorter one.

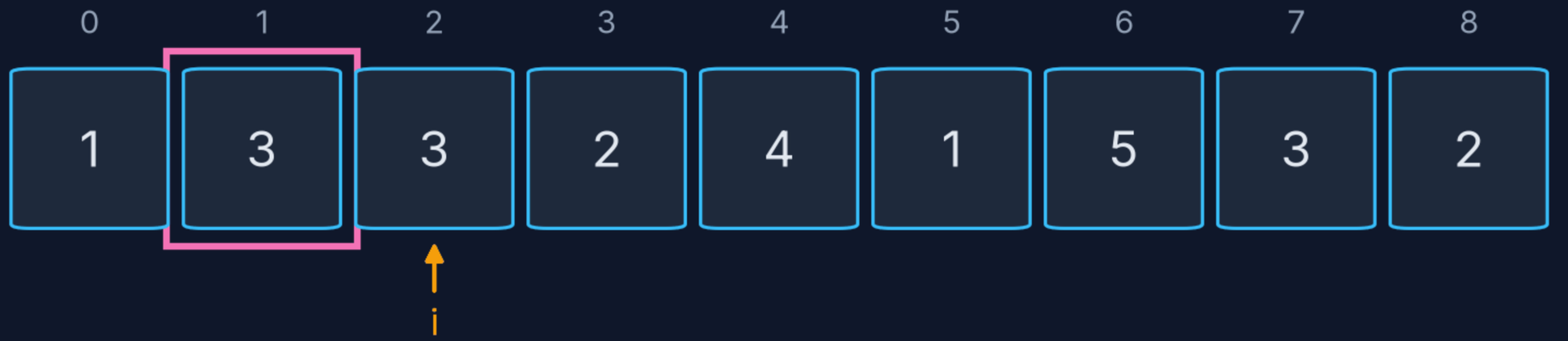
Brute force vs optimal

- Every pair of bars → $O(n^2)$
- For each bar expand left/right → $O(n^2)$
- Monotonic stack → $O(n)$

Approach

- 1. Push indices with increasing heights
- 2. When a shorter building arrives, pop and compute area
- 3. Width spans to the new top
- 4. Flush at the end

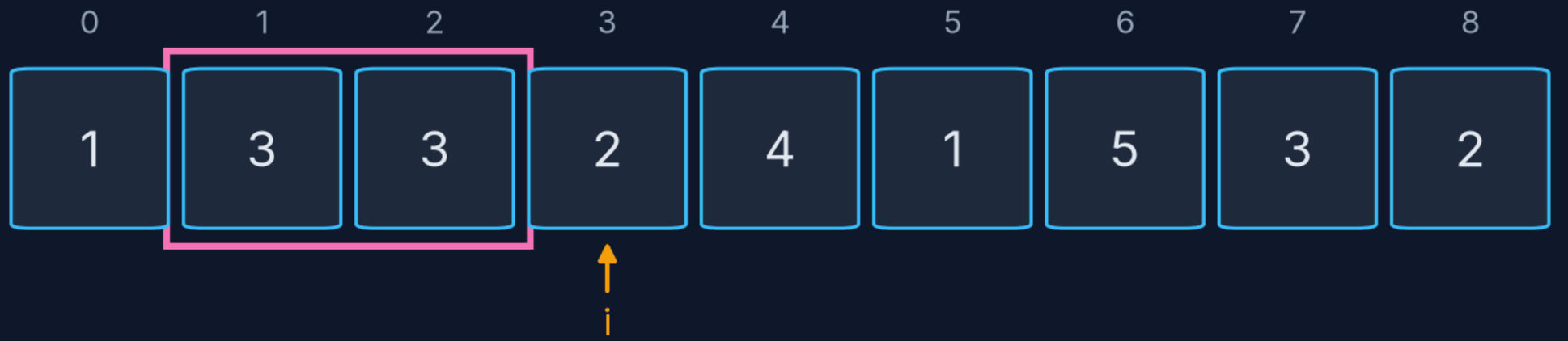
Worked example




```
height 3 × width 1 = 3
best = 3
```

Pop 3: area 3

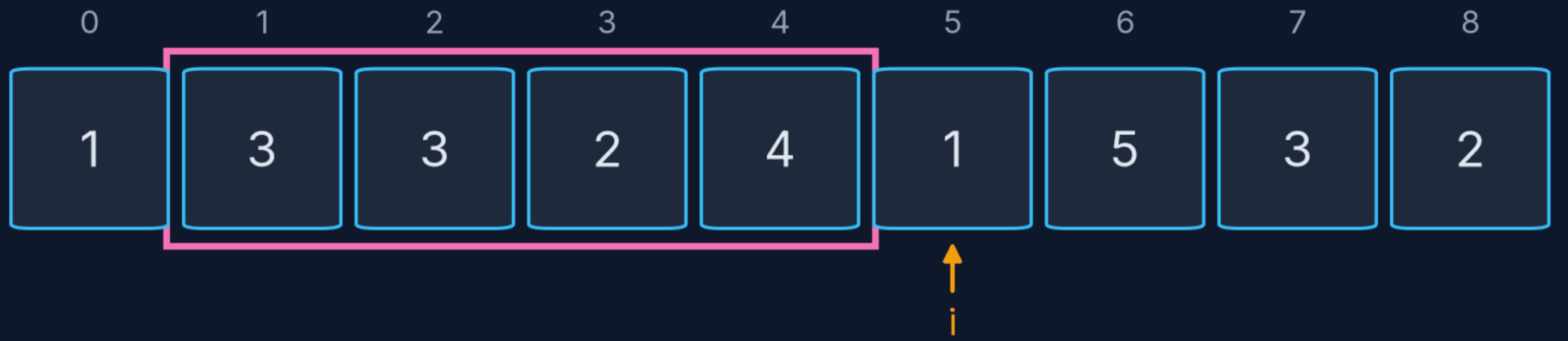
Worked example




```
height 3 × width 2 = 6
best = 6
```

Pop 3: area 6

Worked example




```
height 2 × width 4 = 8
best = 8
```

Pop 2: area 8

Worked example




```
height 1 × width 9 = 9
best = 9
```

Pop 1: area 9

Java solution

```java
public class LargestRectangleUnderSkyline {
    public static int largestRectangleUnderSkyline(int[] h) {
        ArrayDeque<Integer> st = new ArrayDeque<>();
        int best = 0;
        for (int i = 0; i <= h.length; i++) {
            int cur = i == h.length ? 0 : h[i];
            while (!st.isEmpty() && h[st.peek()] >= cur) {
                int height = h[st.pop()];
                int left = st.isEmpty() ? -1 : st.peek();
                best = Math.max(best, height * (i - left - 1));
            }
            st.push(i);
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Sentinel 0 at the end flushes the stack
- Width uses the new top as left boundary
- Equal heights

Interview tips & follow-ups

- Follow-up: maximal rectangle in a binary matrix

Recap

- A monotonic stack finds, for each building, how far it extends left and right before a shorter one.
- Complexity: O(n)